Offline Techniques

CDQ divide & conquer, parallel binary search, offline dynamic connectivity, Mo with updates

Learning goals

- Recognise when reading **all queries first** unlocks a simpler or faster solution
- Use **CDQ divide & conquer** to remove one dimension (3D dominance, dynamic → static, DP)
- Run many binary searches at once with **parallel binary search**
- Handle edge insertions *and deletions* with a **segment tree over time + rollback DSU**
- Extend Mo's algorithm with a **time** dimension for point updates

The offline toolbox

If the problem has ...	Try
points with 3 coordinates, count dominated	CDQ (+ BIT)
updates and queries interleaved, data structure only static	CDQ over time
per query: "first time t when P(t) holds", P monotone	parallel binary search
insert/delete edges, ask connectivity / bipartiteness	segment tree over time + rollback DSU
range queries + point updates, statistic only add/remove-able	Mo with updates

All require: **queries known in advance** (no forced-online decoding).

CDQ divide & conquer: the idea

Elements in a sequence; each element j contributes to later elements i (j < i) under a condition.

```
solve(l, r):
    solve(l, m)
    solve(m, r)
    account for all pairs (j in [l, m), i in [m, r))   ← one dimension is now "free"
```

- The split itself guarantees j < i, removing **one** dimension
- The cross step is a simpler, static problem: often a sort + sweep + BIT
- T(n) = 2T(n/2) + O(n log n) = **O(n log² n)**

3D dominance (Luogu P3810)

Count, for each i, the j ≠ i with $a_j \le a_i$, $b_j \le b_i$, $c_j \le c_i$.

1. Sort by (a, b, c); merge identical triples with a multiplicity `cnt`
2. CDQ on this order → left half has smaller (or equal) a
3. Cross step: both halves sorted by b (merge sort as we return), sweep right half by b, insert left c's into a BIT
4. Equal triples dominate each other: answer for an item = res + cnt − 1

```
dimension a: handled by the split
dimension b: handled by the merge order
dimension c: handled by the BIT
```

CDQ: code

```
// it[l..r) sorted by a on entry; sorted by b on exit.
void cdq(int l, int r) {
    if (r - l <= 1) return;
    int m = (l + r) / 2;
    cdq(l, m); cdq(m, r);
    int i = l, k = l;
    for (int j = m; j < r; j++) {
        while (i < m && it[i].b <= it[j].b) {
            bit.add(it[i].c, it[i].cnt);
            tmp[k++] = it[i++];
        }
        it[j].res += bit.sum(it[j].c);
        tmp[k++] = it[j];
    }
    for (int x = l; x < i; x++) bit.add(it[x].c, -it[x].cnt);  // undo, don't clear
    while (i < m) tmp[k++] = it[i++];
    for (int x = l; x < r; x++) it[x] = tmp[x];
}
```

$n = 10^5$ random triples: ~0.15 s.

Why merging identical items matters

```
items: (1,1,1) (1,1,1)
```

- Without merging, the first copy lands in the left half, the second in the right
- Left → right counted, right → left **never** counted → asymmetric answers
- Lexicographic sort + merge: right can only dominate left if they are equal ⇒ already merged
- General rule: CDQ counts only **left → right**. Make sure "right → left" pairs are impossible or handled

CDQ over time: dynamic → static

Operations: "add w at point (x, y)" and "query sum in rectangle" interleaved.

- Each operation gets a timestamp t: (t, x, y) — **3 dimensions**
- Rectangle query = 4 prefix queries ($x \le X$, $y \le Y$) with signs
- Cross step: updates in left half (earlier) vs queries in right half (later), sweep x, BIT on y
- $O(n \log^2 n)$, memory $O(n)$, no 2D data structure needed

Same trick turns "insertions + queries" into a static problem for many structures (convex hull trick, Li Chao, ...).

CDQ for DP: order matters

dp[i] = f(max / sum of dp[j] over j < i with condition(j, i)) — e.g. 3D LIS.

```
solve(l, r):
    solve(l, m)                          // left half final
    contribute left → right               // right half receives all left values
    solve(m, r)                          // NOW right half can finish
```

- Left, cross, right — **not** left, right, cross
- Cross step must not destroy the order the right recursion expects (re-sort or copy)
- Same pattern: **online convolution** $f_i = \Sigma_{j<i} f_j g_{i-j}$ with NTT in the cross step, $O(n \log^2 n)$ (cf. Lecture 2)

Parallel binary search

Q queries; each asks "smallest k such that P_i(state after k updates) holds", P_i monotone in k.

- One query: binary search, each probe replays k updates → O(M log M)
- Q queries: O(Q M log M) — too slow
- **Batch**: in each round, every unresolved query has a midpoint; replay updates 1..M **once**, and check queries whose midpoint is k right after update k
- log M rounds × O(M + Q) per round

```
round:  for k = 1..M:  apply update k;  for i in bucket[k]:  P_i ? hi = k : lo = k + 1
```

Parallel binary search: code (AGC002 D *Stamp Rally*)

```
vector<int> lo(q, 1), hi(q, m);                  // answer in [lo, hi]
vector<vector<int>> bucket(m + 1);
while (true) {
    bool any = false;
    for (auto& b : bucket) b.clear();
    for (int i = 0; i < q; i++)
        if (lo[i] < hi[i]) bucket[(lo[i] + hi[i]) / 2].push_back(i), any = true;
    if (!any) break;
    DSU d(n);                                    // fresh state each round
    for (int k = 1; k <= m; k++) {
        d.unite(e[k - 1].first, e[k - 1].second);
        for (int i : bucket[k]) {
            if (d.reach(x[i], y[i]) >= z[i]) hi[i] = k;
            else lo[i] = k + 1;
        }
    }
}
```

Query: after adding edges 1..k, do the components of x and y hold ≥ z vertices?

Parallel binary search: requirements

- **Monotone** predicate per query
- A way to **replay** the update sequence from scratch each round (or undo)
- Checking a query at a given time must be cheap (O(log) or O(α))
- Unanswerable queries: search on [1, M + 1] and treat M + 1 as "never"

Classic: POI 2011 *Meteors* (SPOJ METEORS) — each state answers its own "when do I reach my target?", updates are range additions on a circle (BIT with range add).

Offline dynamic connectivity

Operations: add edge, remove edge, query (connected? number of components?). DSU cannot delete.

- Each edge is alive on a time **interval** [t_add, t_remove)
- Build a segment tree over time [0, T); insert each edge into O(log T) nodes covering its interval
- DFS the segment tree: on entering a node, **union** its edges; at a leaf, answer the query; on leaving, **roll back**

```
time:     0   1   2   3   4   5   6   7
edge e:       [=========)                ← stored in nodes covering [1,4)
```

Rollback DSU

```
int find(int x) { while (p[x] != x) x = p[x]; return x; }   // no path compression
void unite(int a, int b) {
    a = find(a); b = find(b);
    if (a == b) { hist.push_back({-1, -1}); return; }      // record the no-op too
    if (s[a] < s[b]) swap(a, b);                           // union by size
    p[b] = a; s[a] += s[b]; comps--;
    hist.push_back({a, b});
}
void rollback() {
    auto [a, b] = hist.back(); hist.pop_back();
    if (a == -1) return;
    p[b] = b; s[a] -= s[b]; comps++;
}
```

- **Path compression breaks rollback** (and its amortisation); union by size alone gives depth ≤ $\log_2 n$
- Every unite pushes exactly one record, so "roll back k times" is always correct

Segment tree over time: code

```
void insertEdge(int node, int l, int r, int ql, int qr, pair<int,int> e) {
    if (qr <= l || r <= ql) return;
    if (ql <= l && r <= qr) { seg[node].push_back(e); return; }
    int m = (l + r) / 2;
    insertEdge(2 * node, l, m, ql, qr, e);
    insertEdge(2 * node + 1, m, r, ql, qr, e);
}
void solve(int node, int l, int r, RollbackDSU& d) {
    for (auto [u, v] : seg[node]) d.unite(u, v);
    if (r - l == 1) { if (isQuery[l]) answer[l] = d.comps; }
    else {
        int m = (l + r) / 2;
        solve(2 * node, l, m, d);
        solve(2 * node + 1, m, r, d);
    }
    for (size_t i = 0; i < seg[node].size(); i++) d.rollback();
}
```


O(T log T) edge insertions, each find O(log n) → **O(T log T log n)**

Variations

- **Bipartiteness** under insert/delete (Codeforces 813F): DSU with parity to the root; an edge closing an odd cycle makes the whole segment-tree subtree "not bipartite"
- **Component sums / sizes**: store aggregates at roots, roll them back too
- **Multi-edges**: match each removal to the most recent unmatched insertion of the same pair
- Online version exists (Holm–de Lichtenberg–Thorup, $O(\log^2 n)$ amortised) — almost never needed in contests
- Alternative offline: LCT maintaining a maximum spanning forest by deletion time (Lecture 4)

Mo's algorithm with updates

Week-34 Mo: queries (l, r), sorted in blocks, O((n + q)√n) pointer moves.

With point updates, add a third pointer **t** = number of updates applied:

- Query = (l, r, t). Sort by (l / B, r / B, t)
- Moving t forward/backward applies/undoes an update; if its position is in [L, R], adjust the statistic
- Block size **B ≈ n^{2/3}** (more precisely $(n^2U/Q)^{1/3}$)
- Total moves: O(Q·B) for l and r, $O(n^2U / B^2)$ for t ⇒ **O(n^{5/3})** when Q, U ~ n

n = q = $1.3 \cdot 10^5$ distinct-count queries with updates: ~0.8 s.

Mo with updates: the moves

```
auto apply = [&](int i, bool forward) {          // cross update i in time
    const Update& u = ups[i];
    int from = forward ? u.oldV : u.newV, to = forward ? u.newV : u.oldV;
    if (L <= u.pos && u.pos <= R) { del(from); add(to); }
    a[u.pos] = to;
};
for (const Query& q : qs) {
    while (t < q.t) apply(t++, true);
    while (t > q.t) apply(--t, false);
    while (L > q.l) add(a[--L]);
    while (R < q.r) add(a[++R]);
    while (L < q.l) del(a[L++]);
    while (R > q.r) del(a[R--]);
    res[q.id] = distinct;
}
```

Store each update's **old** value while reading input (simulate the array once).

Which offline technique?

Technique	Complexity	Needs
CDQ	$O(n \log^2 n)$	contributions decompose into left → right pairs
Parallel binary search	$O((M + Q) \log M \cdot \text{cost})$	monotone per-query predicate, replayable updates
Segment tree over time	$O(T \log T \cdot$ cost of undoable op$)$	operations with **undo**, no deletion
Mo with updates	$O(n^{5/3})$	$O(1)$ add/remove of one element

Many problems admit two of these; pick the one with the smallest, most familiar code.

WF-style signals

- "Answer all queries" with n, q ≤ 10^5 and no forced-online encoding (`x xor lastans`)
- Updates that are hard to delete (DSU, convex hull, linear basis) but easy to undo → segment tree over time
- "For each X, find the first moment ..." → parallel binary search
- Three independent ordering constraints → CDQ
- If the statement **does** encode queries with the last answer, it is deliberately killing these techniques

Pitfalls

- CDQ: clearing the BIT with a full reset (O(n) per call) instead of undoing → O(n²)
- CDQ: ties across the split; forgetting to merge identical items
- CDQ for DP: doing the cross step after the right recursion
- PBS: not resetting the state each round; off-by-one between "after update k" and "before"
- Rollback DSU: path compression; forgetting no-op records
- Segment tree over time: edges never removed must live until T
- Mo with updates: applying an update without checking whether its position is inside [L, R]

Practice problems

- Luogu P3810 三维偏序 (CDQ); Codeforces 848C *Goodbye Souvenir*; Codeforces 1093E *Intersection of Permutations*
- AtCoder AGC002 D *Stamp Rally*; SPOJ METEORS (POI 2011) (parallel binary search)
- Codeforces Gym 100551A *Connect and Disconnect*; Codeforces 813F *Bipartite Checking*
- Library Checker *Dynamic Graph Vertex Add Component Sum*
- Luogu P1903 数颜色; Codeforces 940F *Machine Learning* (Mo with updates)

Summary

- Offline = freedom to reorder; every technique here is a way of reordering
- CDQ removes a dimension by splitting on it: left → right contributions only
- Parallel binary search shares one replay of the updates among all queries per round
- Segment tree over time converts deletions into rollbacks
- Mo with updates adds a time pointer; B ≈ n^{2/3}, O(n^{5/3})